

COMP 364/464

Project 2 Part A: Counting N-Queens Solutions

Term: Fall 2026

Instructor: Melanie Cornelius

Assigned: 30 September 2026

Due: 24 October 2026, 11:59 PM AoE

Description

In this project, you will solve a classic problem: the counting queens. You will write code to count the number of ways to place N queens on an $N \times N$ chessboard so that no two queens attack each other. You're given a working serial program. Your job is to parallelize it with OpenMP, then MPI, then both (if you are in the 464 section), and explain how it performs.

If you would like to [watch a brief and comprehensive review of this problem](#), I recommend the [GeeksForGeeks resources](#).

Part A of this problem involves implementation. Part B will involve performance monitoring.

Serial Implementation

`nqueens.c` is the provided serial implementation of this problem. It counts solutions with a recursive backtracking search. Take a look at the code and comments to see how it works. Utility files are also provided in `utils.c` and `utils.h`. You are welcome to use any of the code from the serial version and utility files when you implement the parallel versions, I especially recommend using the `printout` function for ease of comparison.

Parallel Implementations

You should create multiple parallel implementations of this code, in C, using the below-described strategies. Creating multiple `.c` files is encouraged. Your parallel versions should print the same solution as the serial version, so feel free to borrow from the provided serial code.

Use the following parallelization strategies:

OpenMP two ways: Parallelize the search. You should implement this using both static and dynamic scheduling.

MPI: Break the work into tasks, and divide the tasks among ranks and combine the results (consider `MPI_Reduce`). You may want to try different implementations and task breakdowns; feel free to include multiple MPI implementations in the experiment (described below).

464 Only - Hybrid: Try combining parts 1 and 2. Break down tasks coarsely using MPI and then finer-grained using OpenMPI. This is unlikely to be your most performant option on a single system, but it is representative of real-world HPC programming models.

Experiment: Part 1

1. Choose different values for N, ranging from 4 to 16. For each value of N:
 - a. Time the serial implementation
 - b. Time each of your parallel implementations (using MPI_Wtime): OpenMP static, OpenMP dynamic, MPI (multiple implementations are great but not required), and Hybrid (if you are in COMP 464).
 - c. Vary the number of workers/ranks as much as your hardware allows.
 - d. NOTE: if the runtime of any one test exceeds 40 minutes, record that result as “exceeded walltime” and move on to the next value.
2. Plot these times, showing the scaling of your implementations compared to the serial implementation. You may want to create a different plot for each N value.

Deliverables:

- Source code (multiple files encouraged) and a Makefile
- A README with build and run instructions (Linux or OSX only, please)
- A report explaining how your code works, iterations you went through, and your experimental results.

Grading:

- Correctness: 20%
- Code clarity and README: 20%
- Experiments and plots: 30%
- Report and explanations: 30%

Please remember to follow the course academic honesty and AI policies. If you borrow from existing sources, cite them well, and make sure you fully understand the code you submit. Additionally, you are welcome to use AI tools. Please document your process and model usage, submit that documentation, and make sure you understand your code, as the exams are written and closed-world other than a double-sided sheet of notes (maximum size 8.5”x11”). **Your code *must* be human-readable**, and I recommend commenting on the functionality both for your own understanding and for clarity during grading.

Turn in the assignment as a .zip or .tar on Sakai.